

Hamilton, TSP, VRP và bài toán khó (Hamiltonian Paths, TSP, VRP and Hard Problems)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Sau bài học, người học có thể:

- Phân biệt được Euler, Hamilton, TSP và VRP theo đầu vào, đầu ra và mục tiêu.
- Giải thích được vì sao Hamilton, TSP và VRP là các bài toán khó trong tối ưu tổ hợp.
- Mô hình hóa một tình huống lập tuyến thành đồ thị có trọng số và các ràng buộc tương ứng.
- Cài đặt hoặc mô phỏng được vét cạn/quay lui cho TSP kích thước nhỏ.
- Áp dụng được heuristic gần nhất và 2-opt; biết cách đánh giá chất lượng nghiệm.

Kế hoạch 3 tiết

Thời lượng	Trọng tâm	Hoạt động chính
Tiết 1	Bối cảnh, Hamilton, TSP, độ khó	Phân biệt bài toán; chạy tay ví dụ nhỏ; thảo luận số lượng tour.
Tiết 2	Giải chính xác và heuristic TSP	Quay lui, nearest neighbor, 2-opt; so sánh nghiệm trên cùng dữ liệu.
Tiết 3	VRP và đánh giá thực nghiệm	Mô hình hóa ràng buộc xe; bài tập nhóm; thiết kế báo cáo thực nghiệm.

Nội dung

- 1 Bối cảnh bài toán khó
- 2 Giải chính xác cho TSP kích thước nhỏ
- 3 Heuristic cho TSP
- 4 VRP: mở rộng từ một tour sang nhiều tuyến
- 5 Đánh giá nghiệm và thực nghiệm
- 6 Hoạt động trên lớp và bài tập
- 7 Tổng kết

Tình huống khởi động: giao hàng trong thành phố

Bài toán thực tế

Một kho hàng cần giao cho nhiều điểm trong ngày. Mỗi tuyến đi phải ngắn, đúng thứ tự hợp lý và không vi phạm sức chứa xe.

- Mỗi địa điểm có thể biểu diễn bằng một đỉnh của đồ thị.
- Khoảng cách hoặc thời gian di chuyển là trọng số cạnh.
- Một tour hợp lệ phải đi qua các điểm cần phục vụ theo đúng ràng buộc.
- Câu hỏi thuật toán: nên đi theo thứ tự nào, dùng mấy xe, và đánh giá nghiệm ra sao?

Từ tình huống thực tế sang mô hình đồ thị

Mô hình đồ thị có trọng số

Cho đồ thị $G = (V, E)$, trong đó V là tập địa điểm, E là tập đường đi khả dụng, và c_{ij} là chi phí đi từ i đến j .

- Nếu mọi cặp địa điểm đều có đường đi, ta có đồ thị đầy đủ.
- Nếu $c_{ij} = c_{ji}$, bài toán là đối xứng; nếu không, bài toán là bất đối xứng.
- Chi phí có thể là khoảng cách, thời gian, tiền xăng, hoặc hàm kết hợp nhiều tiêu chí.

Euler khác Hamilton như thế nào?

Tiêu chí	Euler	Hamilton
Đối tượng cần đi qua	Cạnh	Đỉnh
Điều kiện đi qua	Mỗi cạnh đúng một lần	Mỗi đỉnh đúng một lần
Câu hỏi điển hình	Có chu trình Euler không?	Có chu trình Hamilton không?
Mức độ khó	Có tiêu chuẩn bậc đỉnh đơn giản	Không có tiêu chuẩn đơn giản tổng quát

Ghi nhớ: TSP là bài toán tối ưu trên ý tưởng Hamilton: không chỉ cần đi qua mỗi đỉnh một lần, mà còn cần tổng chi phí nhỏ nhất.

Hamilton path và Hamilton cycle

Định nghĩa

Một **Hamilton path** là đường đi qua mỗi đỉnh đúng một lần. Một **Hamilton cycle** là chu trình qua mỗi đỉnh đúng một lần và quay lại đỉnh xuất phát.

- Input: đồ thị $G = (V, E)$.
- Output bài toán quyết định: Có tồn tại Hamilton path/cycle hay không?
- Nếu yêu cầu chỉ trả lời “có/không”, ta đang xét bài toán quyết định.
- Nếu yêu cầu tìm đường đi tốt nhất theo chi phí, ta chuyển sang bài toán tối ưu.

TSP: từ quyết định sang tối ưu

Traveling Salesman Problem

Cho n thành phố và chi phí c_{ij} giữa mỗi cặp thành phố. Tìm chu trình đi qua mỗi thành phố đúng một lần, quay lại điểm xuất phát, sao cho tổng chi phí nhỏ nhất.

$$\min_{\pi} c_{\pi_n \pi_1} + \sum_{t=1}^{n-1} c_{\pi_t \pi_{t+1}},$$

trong đó $\pi = (\pi_1, \dots, \pi_n)$ là một hoán vị của các thành phố.

- Bản quyết định: Có tour chi phí không vượt quá B hay không?
- Bản tối ưu: Tour nào có chi phí nhỏ nhất?

Vì sao bài toán nhanh chóng khó?

Số lượng tour tăng theo giai thừa

Với TSP đối xứng, nếu cố định đỉnh xuất phát và bỏ qua chiều ngược, số tour cần xét là $\frac{(n-1)!}{2}$.

Số đỉnh n	Số tour đối xứng
5	12
10	181,440
15	43,589,145,600
20	60,822,550,204,416,000

Hệ quả: thuật toán vét cạn chỉ phù hợp với ví dụ rất nhỏ.

Phân loại độ khó: cần nói đúng bản chất

- Hamiltonian Cycle là bài toán **NP-complete** ở dạng quyết định.
- TSP ở dạng quyết định là **NP-complete**; TSP ở dạng tối ưu là **NP-hard**.
- VRP là họ bài toán tối ưu lập tuyến, thường chứa TSP như một trường hợp đặc biệt nên cũng rất khó.
- “Khó” không có nghĩa là không giải được; nghĩa là không kỳ vọng có thuật toán chính xác đa thức cho mọi trường hợp tổng quát.

Thông điệp sư phạm

Khi bài toán khó, ta cần kết hợp mô hình hóa, thuật toán chính xác cho kích thước nhỏ, heuristic cho kích thước lớn, và đánh giá thực nghiệm.

Ví dụ trung tâm: 5 địa điểm

Xét 5 địa điểm A, B, C, D, E với ma trận khoảng cách đối xứng:

	A	B	C	D	E
A	0	2	9	10	7
B	2	0	6	4	3
C	9	6	0	8	5
D	10	4	8	0	6
E	7	3	5	6	0

- Cố định điểm xuất phát là A .
- Nhiệm vụ: tìm tour đi qua B, C, D, E đúng một lần rồi quay lại A .

Chạy tay một vài tour

Tour	Chi phí
$A - B - C - D - E - A$	$2 + 6 + 8 + 6 + 7 = 29$
$A - B - D - C - E - A$	$2 + 4 + 8 + 5 + 7 = 26$
$A - C - E - D - B - A$	$9 + 5 + 6 + 4 + 2 = 26$
$A - D - C - B - E - A$	$10 + 8 + 6 + 3 + 7 = 34$

Nhận xét

Với $n = 5$, có thể tính tay. Nhưng chỉ cần n tăng lên 15 hoặc 20, việc liệt kê mọi tour đã trở nên không thực tế.

Ý tưởng

Sinh tất cả các hoán vị của các đỉnh, tính chi phí từng tour và chọn tour tốt nhất.

- Ưu điểm: đơn giản, chắc chắn tìm được tối ưu.
- Nhược điểm: thời gian chạy $O(n!)$.
- Dùng tốt để kiểm chứng thuật toán khác trên bộ dữ liệu nhỏ.
- Trong bài giảng, vết cặn đóng vai trò **baseline chính xác**.

Ý tưởng

Xây dựng tour từng bước. Nếu chi phí tạm thời đã không thể tốt hơn nghiệm tốt nhất hiện có, dừng mở rộng nhánh đó.

- Trạng thái: đỉnh hiện tại, tập đỉnh đã thăm, chi phí tạm thời.
- Quyết định: chọn đỉnh chưa thăm tiếp theo.
- Nghiệm hoàn chỉnh: đã thăm hết các đỉnh và quay lại điểm xuất phát.
- Cắt nhánh đơn giản: nếu chi phí tạm thời $\geq$ nghiệm tốt nhất hiện tại thì bỏ nhánh.

Pseudocode quay lui rút gọn

```
1:  $best \leftarrow +\infty$ ,  $bestTour \leftarrow \emptyset$ 
2: Start from city  $s$ 
3: Backtrack( $current$ ,  $visited$ ,  $cost$ ,  $path$ ):
4: if  $|visited| = n$  then
5:    $total \leftarrow cost + c_{current,s}$ 
6:   if  $total < best$  then
7:     update  $best$  and  $bestTour$ 
8:   end if
9:   return
10: end if
11: for each city  $v \notin visited$  do
12:   if  $cost + c_{current,v} < best$  then
13:     add  $v$  to  $visited$  and  $path$ 
14:     Backtrack( $v$ ,  $visited$ ,  $cost + c_{current,v}$ ,  $path$ )
15:     remove  $v$  from  $visited$  and  $path$ 
16:   end if
17: end for
```

Câu hỏi kiểm tra: Nếu bỏ điều kiện cắt nhánh ở dòng 11, thuật toán còn đúng không? Thời gian chạy thay đổi thế nào?

Ý tưởng

Không chỉ so sánh chi phí đã đi, ta ước lượng thêm chi phí tối thiểu còn lại để quyết định có nên mở rộng nhánh hay không.

- Nếu $cost_so_far + lower_bound \geq best$, nhánh đó không thể cải thiện nghiệm tốt nhất.
- Cận dưới càng chặt thì cắt được càng nhiều nhánh.
- Nhưng tính cận dưới cũng tốn thời gian, nên cần cân bằng.
- Đây là nguyên lý chung trong nhiều bài toán tối ưu tổ hợp.

Ý tưởng nâng cao

Dùng quy hoạch động theo tập con: $DP[S][j]$ là chi phí nhỏ nhất để đi từ s , thăm đúng các đỉnh trong S , và kết thúc ở j .

$$DP[S][j] = \min_{i \in S, i \neq j} \{DP[S \setminus \{j\}][i] + c_{ij}\}.$$

- Thời gian: $O(n^2 2^n)$; bộ nhớ: $O(n 2^n)$.
- Tốt hơn vết cạn về lý thuyết nhưng vẫn tăng theo hàm mũ.
- Có thể giới thiệu như cầu nối sang quy hoạch động nâng cao.

Vì sao cần heuristic?

Lý do thực tế

Khi n lớn, nghiệm chính xác thường quá tốn thời gian. Heuristic tìm nghiệm tốt trong thời gian chấp nhận được.

- Không đảm bảo tối ưu trong mọi trường hợp.
- Thường dùng để có nghiệm ban đầu hoặc nghiệm tham khảo.
- Chất lượng phải được đo bằng thực nghiệm, không chỉ bằng cảm giác.
- Hai heuristic cơ bản trong bài: nearest neighbor và 2-opt.

Ý tưởng

Từ đỉnh hiện tại, luôn đi đến đỉnh chưa thăm có khoảng cách nhỏ nhất.

- 1: Chọn đỉnh xuất phát s ; $current \leftarrow s$
- 2: $tour \leftarrow [s]$, $unvisited \leftarrow V \setminus \{s\}$
- 3: **while** $unvisited \neq \emptyset$ **do**
- 4: Chọn $v \in unvisited$ sao cho $c_{current,v}$ nhỏ nhất
- 5: Thêm v vào $tour$; xóa v khỏi $unvisited$
- 6: $current \leftarrow v$
- 7: **end while**
- 8: Quay lại s

Độ phức tạp: cài đặt đơn giản thường là $O(n^2)$.

Ví dụ nearest neighbor trên 5 địa điểm

Với ma trận đã cho và xuất phát từ A :

- 1 Từ A , gần nhất là B với chi phí 2.
- 2 Từ B , trong các đỉnh chưa thăm C, D, E , gần nhất là E với chi phí 3.
- 3 Từ E , trong các đỉnh chưa thăm C, D , gần nhất là C với chi phí 5.
- 4 Từ C , còn lại D với chi phí 8.
- 5 Quay từ D về A với chi phí 10.

$$A - B - E - C - D - A, \quad \text{cost} = 2 + 3 + 5 + 8 + 10 = 28.$$

So sánh: nghiệm tối ưu ở ví dụ này có chi phí 26, nên nearest neighbor chưa tối ưu.

Nhận xét quan trọng

Cùng một ma trận khoảng cách, chọn điểm xuất phát khác nhau có thể tạo tour khác nhau và chi phí khác nhau.

- Một cải tiến đơn giản: chạy nearest neighbor từ mọi đỉnh xuất phát, lấy tour tốt nhất.
- Khi đó thời gian tăng từ $O(n^2)$ lên khoảng $O(n^3)$.
- Vẫn không có bảo đảm luôn tìm được tối ưu.
- Đây là ví dụ điển hình của đánh đổi giữa thời gian chạy và chất lượng nghiệm.

2-opt: cải thiện nghiệm bằng tìm kiếm cục bộ

Ý tưởng

Nếu tour có hai cạnh “xấu”, ta bỏ hai cạnh đó và nối lại theo cách khác bằng cách đảo một đoạn của tour.

Xét hai cạnh (a, b) và (c, d) trong tour. Phép đổi 2-opt có lợi nếu:

$$c_{ab} + c_{cd} > c_{ac} + c_{bd}.$$

- Nếu điều kiện đúng, thay hai cạnh cũ bằng hai cạnh mới sẽ giảm chi phí.
- Lặp lại đến khi không còn phép đổi nào cải thiện.
- Kết quả là tối ưu cục bộ theo lân cận 2-opt.

Ví dụ một bước 2-opt

Giả sử tour ban đầu là:

$$A - C - D - B - E - A, \quad \text{cost} = 9 + 8 + 4 + 3 + 7 = 31.$$

Chọn hai cạnh (A, C) và (B, E) :

$$c_{AC} + c_{BE} = 9 + 3 = 12, \quad c_{AB} + c_{CE} = 2 + 5 = 7.$$

Vì $12 > 7$, ta đảo đoạn $C - D - B$ để được tour:

$$A - B - D - C - E - A, \quad \text{cost} = 2 + 4 + 8 + 5 + 7 = 26.$$

Bài học

2-opt có thể cải thiện mạnh nghiệm ban đầu, nhưng chỉ đảm bảo dừng ở tối ưu cục bộ.

Pseudocode 2-opt

```
1: Input: một tour ban đầu  $T$ 
2:  $improved \leftarrow true$ 
3: while  $improved$  do
4:    $improved \leftarrow false$ 
5:   for mọi cặp cạnh không kề nhau  $(a, b)$  và  $(c, d)$  trong  $T$  do
6:     if  $c_{ab} + c_{cd} > c_{ac} + c_{bd}$  then
7:       Thực hiện phép đổi 2-opt bằng cách đảo đoạn giữa  $b$  và  $c$ 
8:        $improved \leftarrow true$ 
9:     end if
10:  end for
11: end while
12: Trả về tour  $T$ 
```

Gợi ý cài đặt: biểu diễn tour bằng một danh sách các đỉnh; phép đảo đoạn dùng thao tác reverse trên mảng con.

So sánh ba cách tiếp cận TSP

Cách tiếp cận	Đảm bảo tối ưu	Tốc độ	Vai trò
Vết cận/quay lui	Có	Rất chậm khi n lớn	Baseline chính xác
Nearest neighbor	Không	Nhanh	Nghiem ban đầu
2-opt	Không	Trung bình	Cải thiện nghiệm

Quy trình thực hành khuyến nghị

Với dữ liệu nhỏ: so sánh heuristic với nghiệm tối ưu. Với dữ liệu lớn: so sánh nhiều heuristic, nhiều điểm xuất phát và báo cáo thời gian chạy.

VRP là gì?

Vehicle Routing Problem

VRP tìm các tuyến cho nhiều xe xuất phát từ kho, phục vụ khách hàng, rồi quay lại kho, sao cho tổng chi phí nhỏ và các ràng buộc được thỏa mãn.

- TSP: một người bán hàng, một tour, không xét sức chứa.
- VRP: nhiều xe, nhiều tuyến, có thể có sức chứa, thời gian phục vụ, khung giờ, số lượng xe.
- Nếu chỉ có một xe và không có ràng buộc thêm, VRP trở về gần TSP.

Capacitated VRP: mô hình cơ bản

Dữ liệu vào

Có kho 0, tập khách hàng $V = \{1, \dots, n\}$, nhu cầu q_i của khách hàng i , sức chứa xe Q , và chi phí di chuyển c_{ij} .

- Mỗi khách hàng phải được phục vụ đúng một lần.
- Mỗi tuyến bắt đầu và kết thúc tại kho.
- Tổng nhu cầu trên mỗi tuyến không vượt quá Q .
- Mục tiêu thường là tối thiểu hóa tổng chi phí của tất cả các tuyến.

Ví dụ VRP nhỏ

Kho là O , bốn khách hàng có nhu cầu:

$$q_1 = 2, \quad q_2 = 2, \quad q_3 = 3, \quad q_4 = 1, \quad Q = 4.$$

Một phương án khả thi với 2 xe:

- Xe 1: $O - 1 - 2 - O$, tổng nhu cầu $2 + 2 = 4 \leq Q$.
- Xe 2: $O - 4 - 3 - O$, tổng nhu cầu $1 + 3 = 4 \leq Q$.

Điểm khác TSP

Không chỉ cần thứ tự đi tối ưu; còn phải chia khách hàng vào các xe sao cho thỏa sức chứa.

Tại sao VRP khó hơn TSP?

- Cần quyết định **phân cụm**: khách hàng nào thuộc xe nào.
- Cần quyết định **thứ tự**: mỗi xe đi theo thứ tự nào.
- Cần đảm bảo **ràng buộc**: sức chứa, khung giờ, thời gian phục vụ, số xe.
- Mỗi ràng buộc thực tế thêm vào có thể làm không gian nghiệm tăng mạnh.

Cách nhìn thuật toán

VRP thường được giải bằng heuristic/metaheuristic, hoặc bằng mô hình quy hoạch nguyên cho kích thước vừa và nhỏ.

Heuristic đơn giản cho VRP

Một cách tiếp cận cơ bản:

- ➊ Sắp xếp khách hàng theo khoảng cách đến kho hoặc theo góc cực quanh kho.
- ➋ Gán lần lượt khách hàng vào xe hiện tại nếu còn đủ sức chứa.
- ➌ Khi vượt sức chứa, mở tuyến mới.
- ➍ Tối ưu thứ tự trong từng tuyến bằng nearest neighbor hoặc 2-opt.

Nhận xét

Heuristic này dễ hiểu nhưng có thể tạo nghiệm kém. Nó phù hợp để làm baseline trước khi dùng phương pháp mạnh hơn.

Ý tưởng tiết kiệm

Ban đầu, mỗi khách hàng là một tuyến riêng: $O - i - O$. Nếu ghép hai khách hàng i và j vào cùng tuyến, lượng tiết kiệm xấp xỉ là

$$s_{ij} = c_{0i} + c_{0j} - c_{ij}.$$

- Giá trị s_{ij} càng lớn thì ghép i và j càng đáng cân nhắc.
- Chỉ ghép khi không vi phạm sức chứa và cấu trúc tuyến.
- Đây là một heuristic kinh điển, phù hợp để giới thiệu ở mức ý tưởng.

Đánh giá nghiệm: không chỉ nhìn vào chi phí

Khi báo cáo một thuật toán cho TSP/VRP, cần tối thiểu các thông tin sau:

- **Objective value:** tổng chi phí tour hoặc tổng chi phí các tuyến.
- **Running time:** thời gian chạy theo kích thước dữ liệu.
- **Gap:** độ lệch so với tối ưu hoặc cận dưới nếu có.
- **Feasibility:** nghiệm có thỏa mọi ràng buộc không?
- **Stability:** chạy nhiều lần có cho kết quả ổn định không?

Công thức gap thường dùng

Nếu biết nghiệm tối ưu OPT và thuật toán trả nghiệm có chi phí ALG :

$$gap(\%) = \frac{ALG - OPT}{OPT} \times 100\%.$$

Ví dụ

Trong ví dụ TSP 5 địa điểm, nearest neighbor từ A cho chi phí 28, trong khi tối ưu là 26.

$$gap = \frac{28 - 26}{26} \times 100\% \approx 7.69\%.$$

Thiết kế thực nghiệm nhỏ trên lớp

- 1 Tạo 3 bộ dữ liệu TSP với $n = 6, 8, 10$.
- 2 Với $n = 6, 8$, tính nghiệm tối ưu bằng vét cạn/quay lui.
- 3 Chạy nearest neighbor từ mọi điểm xuất phát.
- 4 Dùng 2-opt cải thiện tour tốt nhất từ nearest neighbor.
- 5 Báo cáo: chi phí, gap, thời gian chạy, và nhận xét.

Mục tiêu học tập

Sinh viên thấy trực tiếp sự khác biệt giữa nghiệm chính xác, nghiệm heuristic và nghiệm sau cải thiện cục bộ.

Mẫu bảng báo cáo kết quả

Dataset	n	Brute force	NN+2-opt	Gap
TSP-1	6	120	124	3.33%
TSP-2	8	215	221	2.79%
TSP-3	10	—	350	—

- Dấu “—” dùng khi không tính được nghiệm tối ưu trong thời gian cho phép.
- Với dữ liệu lớn hơn, có thể so với cận dưới hoặc thuật toán baseline khác.

Sai lầm thường gặp khi làm thực nghiệm

- Chỉ chạy một bộ dữ liệu rồi kết luận tổng quát.
- Không cố định seed khi sinh dữ liệu ngẫu nhiên.
- Không kiểm tra nghiệm có hợp lệ hay không.
- So sánh thời gian chạy nhưng dùng cấu hình máy hoặc ngôn ngữ khác nhau mà không ghi chú.
- Chỉ báo cáo nghiệm tốt nhất, bỏ qua nghiệm trung bình hoặc độ lệch.

Hoạt động 1: phân biệt bài toán

Làm việc nhóm 5 phút

Với mỗi mô tả sau, hãy xác định bài toán gần với Euler, Hamilton, TSP hay VRP.

- 1 Xe quét đường cần đi qua mọi tuyến phố ít nhất một lần.
- 2 Một robot cần kiểm tra mỗi phòng đúng một lần rồi quay về điểm sạc.
- 3 Một nhân viên giao hàng cần đi qua 12 điểm sao cho tổng đường đi ngắn nhất.
- 4 Công ty có 5 xe, mỗi xe chở tối đa 100 kg, cần giao cho 60 khách hàng.

Hoạt động 2: chạy tay nearest neighbor

Yêu cầu

Dùng ma trận 5 địa điểm trong bài, mỗi nhóm chọn một điểm xuất phát khác nhau và chạy nearest neighbor.

- Ghi lại tour tìm được và tổng chi phí.
- So sánh kết quả giữa các điểm xuất phát.
- Trả lời: điểm xuất phát có ảnh hưởng đến nghiệm không?
- Thử dùng một bước 2-opt để cải thiện tour nếu có thể.

Hoạt động 3: mô hình hóa một bài toán VRP

Tình huống

Một kho giao hàng cho 8 khách. Mỗi khách có nhu cầu khác nhau, mỗi xe có sức chứa hữu hạn, và mọi xe phải quay về kho.

Mỗi nhóm cần trả lời:

- 1 Đỉnh, cạnh và trọng số là gì?
- 2 Biên quyết định cần biểu diễn điều gì?
- 3 Ràng buộc nào là bắt buộc, ràng buộc nào là mở rộng?
- 4 Nên dùng thuật toán chính xác hay heuristic? Vì sao?

Bài tập củng cố

- 1 Chứng minh rằng với TSP đối xứng, sau khi cố định điểm xuất phát, số tour khác nhau là $(n - 1)!/2$.
- 2 Viết chương trình sinh tất cả tour TSP cho $n \leq 10$ và trả về tour tốt nhất.
- 3 Cài đặt nearest neighbor từ mọi điểm xuất phát và so sánh với nearest neighbor từ một điểm cố định.
- 4 Cài đặt một vòng lặp 2-opt và kiểm tra nghiệm sau mỗi lần cải thiện.
- 5 Tạo một ví dụ VRP nhỏ có sức chứa Q và chỉ ra ít nhất hai phương án khả thi.

Bảng tổng kết

Bài toán	Câu hỏi chính	Ghi nhớ
Euler	Đi qua cạnh	Có tiêu chuẩn bậc đỉnh rõ ràng trong nhiều trường hợp.
Hamilton	Đi qua đỉnh	Bài toán quyết định khó.
TSP	Tour Hamilton chi phí nhỏ nhất	Tối ưu tổ hợp; thường cần heuristic khi dữ liệu lớn.
VRP	Nhiều tuyến có ràng buộc	Gần thực tế hơn TSP nhưng khó hơn do thêm phân tuyến và ràng buộc.

- TSP/VRP là cầu nối tốt giữa lý thuyết thuật toán và tối ưu ứng dụng.
- Bài toán khó không chỉ cần “code chạy được”, mà cần mô hình đúng và đánh giá nghiêm túc.
- Thuật toán chính xác phù hợp với dữ liệu nhỏ và vai trò kiểm chứng.
- Heuristic như nearest neighbor và 2-opt phù hợp để tạo nghiệm nhanh, nhưng cần đo gap và thời gian chạy.
- VRP cho thấy chỉ một ràng buộc thực tế như sức chứa xe cũng làm bài toán phức tạp hơn đáng kể.

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*. Chương về NP-completeness và quy hoạch động.
- Kleinberg, Tardos. *Algorithm Design*. Chương về local search và approximation.
- Applegate, Bixby, Chvátal, Cook. *The Traveling Salesman Problem: A Computational Study*.
- Tài liệu học thuật của Waterloo TSP và Concorde TSP Solver.
- NetworkX documentation: approximation algorithms for TSP and routing-related problems.